

7.3 全志H616智能家居系统

首先，这个项目的课程要吸收到位，在吸收到位的基础上，为了卷面试，下文会有些提升的东西给大家。

这个项目一定要引导面试官的注意力到项目框架，强调工厂模式，而关于点灯控制也要引导面试官注意力到字符设备驱动的开发，课程里有讲，要去复习。

然后一个灯有可能即被语音控制，也被网络控制，可以引导面试官聊一下多线程同步中的信号量，下文也有教如何阐述。

如果聊人脸识别，根据临场面试官情况，算法的话他也不会，主要看我们用什么方案，一种是openCV的，我把原理和代码放下面，另一种是课程中

阿里云的OCR识别。

如果有讲视频监控，可以说因为时间比较赶，用mjpeg-streamer先实现，目前在筹备学习openCV和ffmpeg的音视频知识，也买了瑞芯微的rv1126来学习

关于音视频的SDK，这样来获取一些好感。（这部分课程确实在规划中，目前短期要面试的同学可以这样说一说）

7.3.1 你说下这个项目是怎么做的

先说项目的涉及框架：

项目主要框架采用多线程并发下的简单工厂模式进行管理，用C语言的结构体实现面向对象的编程思想。

框架分为设备管理工厂，输入控制工厂，和传感器工厂。

在工厂模式的架构之下，每个外设的开发都进行模块化分离，一个代码文件维护一个功能或者设备。

设备工厂主要实现了对设备控制的管理，有灯光，风扇，门锁，摄像头，有的设备是通过GPIO口继电器直连，有的是通过网络对接到单片机控制。

输入控制工厂支持语音识别的输入，网络远程数据接入

传感器工厂集成了温湿度，人体红外和火灾报警节点模块。

接着说设备管理工厂

设备有两种，一种是全志H616的GPIO直连，对GPIO控制用全志平台自带的wringpi库调试，后期自己实现了字符设备驱动程序，完成设备节点注册，打开设备，

关闭设备，设备数据交互功能并把设备驱动注册到操作系统中。对简单字符设备框架的开发和流程有一定的了解。

另一种设备是基于网络通信的，在主程序的一条线程中构建一个socket服务器，基于TCP连接的，用于MCU的通信，比如有的设备是通过C51外接wifi模块后通过

TCP客户端接入。

接下来说输入控制工厂

支持语音识别，因为数学能力不足，无法在短期内研究并开发语言识别算法，采用的是基于串口通信的su-06模块，主要采集语音模块识别后的串口数据，

主要是Linux应用层串口的开发，同时输入控制工厂中也构建了一个socket服务器，和设备管理工厂的socket服务器用不同的端口号区分开来，这样就支持

手机app，QT上位机，微信小程序等接口的访问。

最后是传感器工厂

支持温湿度数据获取（这里如果对全志H616Linux温湿度获取表述没有信心，可以把面试官引导到单片机上，C51或者STM32单片机获取温度后把数据通过

网络传输给Linux的方式），支持人体红外检测，人体红外检测也是一个数据量，采用H616Linux的GPIO口来判断是否感应到人，火灾报警的节点可以是全志

GPIO也可以是单片机通过网络来把数据上报。

人体红外是为人脸识别开锁做的唤起功能，当识别到有人到来，就启动摄像头并进行人脸识别

上述陈述完后，面试官可能会针对具体功能点，提出问题。所以项目要掌握到位，常规性技术问题，自己临场发挥。以下预设了一些问题：

7.3.2 面试官问如何实现人脸识别的

非常有信心的告诉你，如果面试官年纪不大，或者不是专门研究人脸识别的，他也不会。这个时候看你要不要装逼了。

如果想要装逼：就把这段话记住！！！！

首先，使用人脸检测算法，如Haar特征分类器或深度学习模型，来检测图像中的人脸。

然后，对检测到的人脸进行特征提取，常用的方法包括局部二值模式（LBP）特征提取。

接下来，将提取的特征向量用于比对，

这可以通过计算两个特征向量之间的相似度来完成，常用的相似度度量方法包括欧氏距离或余弦相似度。

最后，根据相似度结果判断两个人脸是否为同一人。

在Linux环境下，这些步骤可以通过调用相应的库和API来实现，例如使用OpenCV进行图像处理和特征提取，

以及使用深度学习框架进行人脸检测和特征比对。

示例代码：代码也容易，看完后理解一下。需要一个比对的图片，还有一个现场使用opencv拍摄的照片，还需要两个模型文件，模型文件可以说是网上找的。

代码使用OpenCV库从摄像头捕获实时视频流，并在检测到人脸时拍照生成JPG文件，然后与另一张JPG文件进行人脸比对：

```
#include <opencv2/opencv.hpp>
#include <opencv2/face.hpp>
#include <vector>
#include <iostream>

using namespace cv;
using namespace cv::face;

// 函数：计算两个特征向量之间的余弦相似度
double calculateCosineSimilarity(const std::vector<float>& face_encoding1, const
std::vector<float>& face_encoding2) {
```

```

double dot_product = 0.0;
for (size_t i = 0; i < face_encoding1.size(); i++) {
    dot_product += face_encoding1[i] * face_encoding2[i];
}
double norm_product1 = 0.0, norm_product2 = 0.0;
for (size_t i = 0; i < face_encoding1.size(); i++) {
    norm_product1 += face_encoding1[i] * face_encoding1[i];
    norm_product2 += face_encoding2[i] * face_encoding2[i];
}
double cosine_similarity = dot_product / (sqrt(norm_product1) *
sqrt(norm_product2));
return cosine_similarity;
}

// 函数：从摄像头捕获人脸并保存为JPG文件
bool captureFaceFromCamera(const std::string& filename, CascadeClassifier&
face_cascade) {
    VideoCapture capture(0);
    if (!capture.isOpened()) {
        std::cerr << "Unable to open camera" << std::endl;
        return false;
    }

    Mat frame, grayFrame;
    while (true) {
        capture >> frame;
        if (frame.empty()) {
            std::cerr << "Blank frame grabbed" << std::endl;
            return false;
        }

        cvtColor(frame, grayFrame, COLOR_BGR2GRAY);
        std::vector<Rect> faces;
        face_cascade.detectMultiScale(grayFrame, faces, 1.1, 3, 0 |
CASCADE_SCALE_IMAGE, Size(30, 30));

        for (size_t i = 0; i < faces.size(); i++) {
            rectangle(frame, faces[i], Scalar(255, 0, 0), 2);
            if (i == 0) { // 只处理第一个检测到的人脸
                Mat faceROI = grayFrame(faces[i]);
                imwrite(filename, faceROI);
                return true;
            }
        }
    }
    return false;
}

int main() {
    // 加载人脸分类器
    CascadeClassifier face_cascade;
    face_cascade.load("haarcascade_frontalface_default.xml"); // 确保这个文件路径是
正确的

    // 从摄像头捕获人脸并保存为JPG文件
    std::string filename1 = "face1.jpg";

```

```

std::string filename2 = "face2.jpg";
if (!captureFaceFromCamera(filename1, face_cascade)) {
    std::cerr << "Failed to capture face from camera" << std::endl;
    return -1;
}

// 加载另一张人脸图像
Mat image2 = imread(filename2, IMREAD_GRAYSCALE);
if (image2.empty()) {
    std::cerr << "Failed to load " << filename2 << std::endl;
    return -1;
}

// 创建LBPH人脸识别器
Ptr<LBPHFaceRecognizer> model = LBPHFaceRecognizer::create();
model->read("face_recognizer.yml"); // 假设你已经训练并保存了模型

// 计算两张人脸的特征向量
std::vector<float> face_encoding1 = model->getLabelsFor
Faces(imread(filename1, IMREAD_GRAYSCALE))[0];
std::vector<float> face_encoding2 = model->getLabelsFor Faces(image2)[0];

// 计算余弦相似度
double similarity = calculateCosineSimilarity(face_encoding1,
face_encoding2);

// 根据相似度判断是否为同一个人
if (similarity > 0.5) { // 假设阈值为0.5
    std::cout << "The two faces belong to the same person." << std::endl;
} else {
    std::cout << "The two faces do not belong to the same person." <<
std::endl;
}

return 0;
}

```

在这个代码中，`captureFaceFromCamera` 函数负责从摄像头捕获人脸，并将其保存为JPG文件。

然后，主函数使用这个JPG文件与另一张JPG文件进行人脸比对。

请确保您已经训练了LBPH人脸识别器，并且有一个有效的模型文件 `face_recognizer.yml`。

此外，您需要将 `"haarcascade_frontalface_default.xml"` 替换为您的实际文件路径，

这个文件是OpenCV提供的用于人脸检测的预训练模型。您可以从OpenCV的GitHub仓库或其他资源下载这个文件。

如果不想装逼，那就用项目中的实现方式OCR识别，按照以下方式陈述

开发流程涉及使用阿里云OCR识别服务来验证拍照后的人员是否为有效人员，

大致步骤如下：首先，在阿里云控制台开通OCR服务，并获取相应的AccessKey ID和AccessKey Secret。

然后，集成阿里云提供的SDK到项目中，并配置好相关参数，包括区域ID和API endpoint。

接着，编写代码调用阿里云OCR接口，上传拍摄的人脸照片，并接收识别结果。

根据返回的识别结果，你可以获取到人脸特征信息，进而与数据库中存储的有效人员信息进行比对。

最后，根据比对结果判断是否为有效人员，并执行相应的业务逻辑。

这个过程中，确保传入的照片满足阿里云的图片格式要求，以提高识别准确性。

7.3.3 如何处理资源竞争的

比如，一个灯同时收到手机APP和微信小程序的信号，这种情况怎么处理。（我们项目不一定考虑这个场景，但有可能会问，也可以借题发挥展示自己对Linux的了解）

这样的问题有可能是面试官问，也可以自己发起，来展示自己。

可以这么回答：

在智能家居项目中，当手机APP和微信小程序需要同时控制同一个智能灯时，

为了避免资源竞争问题，

在Linux系统中使用线程信号量（semaphore）来实现同步控制。

具体解决方案是：首先定义一个信号量变量，并在系统启动时对其进行初始化，设置为1，表示灯的状态是互斥的。

当任一控制端（APP或小程序）请求控制灯时，它首先尝试获取信号量，如果成功获取，则执行灯的开关操作；操作完成后，释放信号量，

允许其他控制端进行操作。如果信号量已被占用，

则当前请求会被阻塞，直到信号量被释放。

这样，通过信号量的P（等待）和V（信号）操作，确保了在任何时刻只有一个控制端能对灯进行操作，从而避免了因并发访问而导致的资源竞争问题。

7.3.4 面试官刁难，智能家居项目垃圾

是的，智能家居项目确实做的人多了，导致大家对它有偏见。作为行业新人，这个项目不是目的，主要是用来整合我对C语言，单片机，Linux系统以及全志平台的

技术掌握，来锻炼我软硬件开发协调能力。比如在做项目架构的时候，工厂模式让我更加确定项目设计的重要性，能够把每个功能具体到一个文件中，非常方便

开发分工及开发调试，对代码的稳定性可拓展性都很有帮助，因为输入端对设备的控制是多线程，多样的，难免设计资源竞争问题，刚好把我研究的线程同步比如

信号量，条件控制等都应用上去了。然后也是为了不会忘记早前学的单片机知识，把一些设备的控制放在C51或者STM32端，我觉得这是一个相对综合的项目，

对入行之后的开发任务，有很多技术是可以平移的。对了，其实也怕大佬你说项目垃圾，所以做项目过程中关于Linux设备驱动我都尽量自己写，不用现成的，

比较遗憾的是视频监控功能我想要去了解视频，音频编解码这块，时间还没来得及，不然我相信今天面试的话能让我更有把握。

7.3.5 用队列优化

因为现在面试内卷，为了体现我们对Linux系统编程的掌握情况，可在合理的时机抛出优化方案，用到消息队列。

在我们的项目中，对于APP和语音控制输入的话用一个工厂模式管理，

不同的灯，风扇等设备用另外一种工厂模式管理，这两个工厂之间不直接控制，而是通过消息队列维护，代码思路可以这样实现

面试官问消息队列的细节，就是代码中mq_open打开队列，mq_send往队列发消息，mq_receive读取队列消息

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <mqueue.h>

// 定义消息结构体
typedef struct {
    int command_type; // 命令类型
    int device_type;  // 设备类型
    int device_id;    // 设备ID
    int parameter;    // 参数
} ControlMessage;

// 定义队列名称
#define QUEUE_NAME "/HomeAutomationQueue"

// 发送控制命令到队列
int send_control_command(mqd_t mq, ControlMessage msg) {
    if (mq_send(mq, (char *)&msg, sizeof(ControlMessage), 0) == -1) {
        perror("mq_send failed");
        return -1;
    }
    return 0;
}

// 从队列接收控制命令
ControlMessage receive_control_command(mqd_t mq) {
    ControlMessage msg;
    ssize_t bytes_read = mq_receive(mq, (char *)&msg, sizeof(ControlMessage),
    NULL);
    if (bytes_read == -1) {
        perror("mq_receive failed");
        exit(EXIT_FAILURE);
    }
    return msg;
}

int main() {
    // 创建或打开队列
    mqd_t mq = mq_open(QUEUE_NAME, O_CREAT | O_RDWR);
    if (mq == (mqd_t)-1) {
        perror("mq_open failed");
        exit(EXIT_FAILURE);
    }
}
```

```

}

// 设置队列属性
struct mq_attr attr;
attr.mq_flags = 0;
attr.mq_maxmsg = 10;
attr.mq_msgsize = sizeof(ControlMessage);
attr.mq_curmsgs = 0;
if (mq_setattr(mq, &attr, NULL) == -1) {
    perror("mq_setattr failed");
    exit(EXIT_FAILURE);
}

// 发送控制命令的示例
ControlMessage msg = {1, 2, 3, 4}; // 假设的命令类型、设备类型、设备ID和参数
send_control_command(mq, msg);

// 接收控制命令的示例
msg = receive_control_command(mq);

// 处理接收到的控制命令
printf("Received command: %d %d %d %d\n", msg.command_type, msg.device_type,
msg.device_id, msg.parameter);

// 关闭队列
mq_close(mq);

return 0;
}

```